

Misioneros y Caníbales

BFS, DFS y DFS Recursivo — Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Problema	2
1.1. Representación del estado	2
1.2. Restricciones de validez	2
2. Movimientos Posibles	2
3. BFS — Búsqueda por Amplitud	2
4. DFS — Búsqueda por Profundidad con Pila	3
5. DFS Recursivo	3
6. Reconstrucción del Camino	4
7. Interfaz Gráfica	4

1. Descripción del Problema

El problema de **Misioneros y Caníbales** es un rompecabezas clásico de IA. Tres misioneros y tres caníbales deben cruzar un río en un bote con capacidad máxima de 2 personas, cumpliendo la restricción de que los caníbales nunca superen en número a los misioneros en ninguna orilla.

1.1. Representación del estado

Cada estado se codifica como un vector de 5 enteros:

$$[m_{izq}, c_{izq}, bote, m_{der}, c_{der}]$$

- **Estado inicial:** $[3, 3, 0, 0, 0]$
- **Estado meta:** $[0, 0, 1, 3, 3]$
- **bote:** 0 = orilla izquierda, 1 = orilla derecha

1.2. Restricciones de validez

Un estado es válido si:

1. Todos los valores son ≥ 0 y ≤ 3 .
2. Si $m_{izq} > 0$, entonces $m_{izq} \geq c_{izq}$.
3. Si $m_{der} > 0$, entonces $m_{der} \geq c_{der}$.

2. Movimientos Posibles

Desde cada estado se generan hasta 5 movimientos posibles (lo que cabe en el bote):

Misioneros	Caníbales
1	0
2	0
0	1
0	2
1	1

3. BFS — Búsqueda por Amplitud

BFS usa una **cola FIFO** para explorar los estados nivel por nivel, garantizando la solución óptima (menor número de pasos).

```
1 Nodo_BFS resolver() {
2     cola.encolar(estadoInicial);
3     visitados.add(estadoInicial);
4
5     while (!cola.estaVacia()) {
6         Nodo_BFS actual = cola.desencolar();
7         if (esSolucion(actual)) return actual;
```

```

8
9         for (Nodo_BFS hijo : generarHijos(actual)) {
10             if (!estaVisitado(hijo)) {
11                 visitados.add(hijo);
12                 cola.encolar(hijo);
13             }
14         }
15     }
16     return null;
17 }

```

Resultado: solución óptima en **11 pasos**, 15 nodos generados.

4. DFS — Búsqueda por Profundidad con Pila

DFS usa una **pila LIFO** para explorar el camino más profundo primero. No garantiza la solución óptima pero usa menos memoria que BFS.

```

1  Nodo_DFS resolver() {
2      pila.apilar(estadoInicial);
3      visitados.add(estadoInicial);
4
5      while (!pila.estaVacia()) {
6          Nodo_DFS actual = pila.desapilar();
7          if (esSolucion(actual)) return actual;
8
9          for (Nodo_DFS hijo : generarHijos(actual)) {
10             if (!estaVisitado(hijo)) {
11                 visitados.add(hijo);
12                 pila.apilar(hijo);
13             }
14         }
15     }
16     return null;
17 }

```

5. DFS Recursivo

Implementación recursiva del DFS. Más elegante en código pero con riesgo de **StackOverflowError** en problemas más profundos.

```

1  Nodo_DFS resolverRecursivo(Nodo_DFS nodo,
2                             Set<String> visitados) {
3      if (esSolucion(nodo)) return nodo;
4
5      for (Nodo_DFS hijo : generarHijos(nodo)) {
6          String clave = estadoAClave(hijo);
7          if (!visitados.contains(clave)) {
8              visitados.add(clave);
9              Nodo_DFS resultado =

```

```

10         resolverRecursoivo(hijo, visitados);
11         if (resultado != null) return resultado;
12     }
13 }
14 return null;
15 }

```

6. Reconstrucción del Camino

Cada nodo guarda una referencia al nodo padre. Para reconstruir el camino desde la solución hasta el estado inicial se recorre la cadena de padres:

```

1 List<int[]> reconstruirCamino(Nodo_BFS solucion) {
2     List<int[]> camino = new ArrayList<>();
3     Nodo_BFS nodo = solucion;
4     while (nodo != null) {
5         camino.add(0, nodo.getEstado());
6         nodo = nodo.getPadre();
7     }
8     return camino;
9 }

```

7. Interfaz Gráfica

La clase `InterfazCyM` implementa:

- Representación visual de ambas orillas con personajes dibujados en `paintComponent()`.
- Botones para mover 1M, 2M, 1C, 2C ó 1M+1C manualmente.
- Botón “**Resolver (BFS)**” que ejecuta BFS y anima el camino solución con un `javax.swing.Timer` de 700ms entre pasos.